University of Frontier Technology, Bangladesh
Department of Internet of Things and Robotics Engineering
Course: Cloud Computing Sessional (CC 484)

PROJECT REPORT

ZenBot v2

Serverless AI Knowledge Platform

Submitted in Partial Fulfillment of the Requirements for
Cloud Computing Sessional (CC 484)

Submitted By:
1. **MD Naimur Rashid (2101042)** (1st)
2. Sourav Chakraborty (2101031)
3. MD Rohan Mondal (2101041).

Group:02
Department of Internet of Things and Robotics Engineering University of Frontier Technology, Bangladesh

Submitted To:
- Farzana Akter

Assistant Professor
Department of Internet of Things and Robotics Engineering University of Frontier Technology, Bangladesh

Table of Contents

Abstract 5
Chapter 1: Introduction 6
1.1 Background of AI Chatbots 6
1.2 Role of Cloud Computing in Modern Applications 6
1.3 Importance of Serverless Architecture 7
1.4 Motivation Behind Building ZenBot v2 8
1.5 Objectives of the Project 9
1.6 Scope of the System 9
1.7 Overview of the Project 10
Chapter 2: Literature Review and Background 11
2.1 What Are Chatbots? 11
2.2 Evolution of AI Assistants 11
2.3 Serverless Computing Concepts 12
2.4 Cloud Computing Architecture 13
2.5 Benefits of Serverless Systems 13
2.6 Educational AI Assistants 14
2.7 Knowledge Retrieval Systems 14
2.8 Technologies Used 15
Chapter 3: System Methodology 16
3.1 Overall System Architecture 16
3.2 Frontend Architecture 16
3.3 Backend Architecture 17
3.4 API Communication Flow 17
3.5 Database Structure 18
3.6 Modular Design Approach 19
Chapter 4: Implementation 20
4.1 AWS Services Used in This Project 20
4.1.1 Amazon S3 — Static Website Hosting 20
4.1.2 AWS Lambda — Serverless Compute 21
4.1.3 Amazon API Gateway — HTTP API 22
4.1.4 Amazon DynamoDB — NoSQL Database 22
4.1.5 AWS IAM — Permission Management 23

4.1.6 AWS Lambda Layers — Dependency Management 24
4.1.7 Amazon Cognito — Authentication Architecture 24
4.2 Frontend Interface Implementation 25
4.2.1 Chatbot UI 25
4.2.2 Login System 26
4.2.3 Session Management 26
4.3 Backend Logic Implementation 26
4.4 Wikipedia Knowledge Retrieval Module 27
4.5 PDF Processing Module 27
4.6 Chat Session System 28
4.7 User Management System 29
Chapter 5: System Workflow 30
5.1 Complete User Journey Walkthrough 30
5.2 Application Access and Frontend Loading 30
5.3 Authentication Flow 30
5.4 Chat Interaction Workflow 30
5.5 PDF Question Answering Workflow 31
5.6 Session History Retrieval 32
Chapter 6: ZenBot v1 vs ZenBot v2 — Comparative Analysis 33
6.1 Introduction to the Comparison 33
6.2 Interface Quality 34
6.3 Cloud Architecture 34
6.4 Features 35
6.5 Knowledge Capability 35
6.6 Scalability 35
Chapter 7: Advantages of the System 37
7.1 Scalable Architecture 37
7.2 Serverless Cost Efficiency 37
7.3 Modular Design 38
7.4 Educational Usefulness 38
7.5 Cloud-Native Deployment 38
7.6 Improved User Experience 39
Chapter 8: Limitations of the System 40
8.1 Limited AI Reasoning Capability 40

8.2 Simple Keyword Matching for PDF QA 40
8.3 Dependence on Wikipedia for External Knowledge 40
8.4 Basic Authentication Security 40
8.5 No Real-Time Communication 41
8.6 Lambda Cold Start Latency 41
Chapter 9: Future Improvements 42
9.1 Vector Search and Semantic Retrieval 42
9.2 Retrieval-Augmented Generation (RAG) Architecture 42
9.3 Real LLM Integration 43
9.4 Improved Authentication with Amazon Cognito 43
9.5 Mobile-First Responsive Design 43
9.6 Advanced Session Context Management 44
9.7 Analytics and Usage Dashboard 44
Chapter 10: Conclusion 45
10.1 Summary of Achievements 45
10.2 Demonstration of Serverless Cloud Architecture 45
10.3 Learning Outcomes 46
10.4 Importance of Practical Cloud Computing Projects 46
References 48

Abstract

ZenBot v2 is a cloud-native, serverless AI-powered knowledge platform developed as an academic project under the Cloud Computing Sessional (CC 484) course at the University of Frontier Technology, Bangladesh. The platform is built entirely upon Amazon Web Services (AWS) and provides users with an intelligent conversational interface capable of retrieving factual information from Wikipedia, answering questions based on uploaded educational PDF documents, and maintaining persistent session-based chat history. The system is an improved successor to an earlier prototype called ZenBot AI v1, which employed a monolithic server architecture with limited scalability and functionality.

ZenBot v2 demonstrates the practical application of serverless cloud computing principles in the domain of educational AI platforms. The architecture leverages Amazon S3 for static frontend hosting, AWS Lambda for serverless backend computation, Amazon API Gateway for HTTP API routing, Amazon DynamoDB for NoSQL data persistence, AWS IAM for security management, and AWS Lambda Layers for dependency management. The system supports multi-session chat management, user login and profile management, login history tracking, Wikipedia-based knowledge retrieval, and PDF document processing with question-answering capability.

This report presents a detailed account of the system design, architecture, implementation, module-level explanation, and comparative analysis between v1 and v2. The outcome of this project validates the effectiveness of serverless cloud architecture in delivering scalable, cost-efficient, and feature-rich educational AI applications without requiring traditional server infrastructure.

Chapter 1: Introduction

1.1 Background of AI Chatbots

The emergence of conversational artificial intelligence has been one of the most transformative technological developments of the past two decades. Chatbots, defined as software applications designed to simulate human conversation through text or voice interfaces, have evolved from simple rule-based scripts to sophisticated language-processing systems capable of understanding context, intent, and nuanced natural language queries. Early iterations of chatbot systems, such as ELIZA developed at MIT in the 1960s, operated on pattern-matching principles and offered no real intelligence beyond scripted response templates. These systems, while pioneering, demonstrated fundamental limitations in adaptability and practical utility.

The proliferation of the internet and the subsequent increase in computing power enabled a new generation of chatbots in the late 1990s and early 2000s. Applications such as SmarterChild on AOL Instant Messenger demonstrated that conversational agents could be deployed at scale and serve millions of users for entertainment and information retrieval purposes. The subsequent decade witnessed the integration of natural language processing (NLP) techniques, making chatbots more context-aware and capable of understanding loosely structured natural language queries.

A major turning point arrived with the introduction of deep learning and transformer-based language models. The release of attention-based neural architectures, culminating in models such as BERT, GPT, and their successors, fundamentally altered the landscape of conversational AI. These models, trained on billions of tokens of text, demonstrated emergent capabilities in reasoning, summarization, translation, and knowledge-intensive QA tasks. Today, AI chatbot platforms power customer service portals, educational tools, healthcare assistants, legal advisory systems, and entertainment applications across every industry. The global chatbot market, valued at several billion US dollars, continues to expand rapidly as businesses recognize the efficiency and scalability benefits that automated conversational interfaces provide.

1.2 Role of Cloud Computing in Modern Applications

Cloud computing has become the foundational infrastructure for virtually all modern digital applications. The transition from on-premises hardware to cloud-based compute resources has

provided organizations with unprecedented flexibility, scalability, and cost efficiency. Cloud providers such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform have democratized access to enterprise-grade computing infrastructure, enabling even small academic teams to deploy globally available applications with professional reliability and uptime guarantees.

In the context of AI applications, cloud computing is not merely a deployment convenience but a fundamental requirement. Training large AI models, processing vast datasets, and serving real-time inference requests to thousands of simultaneous users demand elastic compute resources that cannot be economically provisioned using traditional fixed hardware. Cloud platforms provide the ability to scale compute resources dynamically in response to demand, ensuring consistent performance without over-provisioning.

For academic projects and startups, cloud computing removes the barrier of capital expenditure associated with physical servers. The pay-as-you-go model allows developers to build and deploy production-grade systems with minimal upfront investment, a factor particularly important in university settings where resources are constrained. AWS, as the dominant cloud provider with over 200 fully featured services, offers an ecosystem that supports every aspect of application development from compute and storage to machine learning and security.

1.3 Importance of Serverless Architecture

Serverless computing represents a paradigm shift in how applications are designed, deployed, and operated. In a traditional server-based model, developers must provision virtual machines or physical servers, manage operating system configurations, install runtime dependencies, monitor server health, and scale instances manually or through complex auto-scaling policies. This operational overhead consumes significant engineering effort and can become a bottleneck in rapid application development cycles.

Serverless architecture, epitomized by AWS Lambda, abstracts away all infrastructure management concerns. Developers write business logic in the form of discrete functions, which are uploaded to the cloud platform. The platform is responsible for provisioning compute resources, executing the function in response to triggers, scaling automatically to accommodate concurrent requests, and decommissioning resources when execution completes. This model

eliminates the concept of idle compute resources, as functions are only executed when triggered, leading to highly efficient resource utilization.

From an economic standpoint, serverless pricing is based on the number of function invocations and the duration of execution, making it exceptionally cost-effective for applications with variable or unpredictable traffic patterns. For an educational platform such as ZenBot v2, where usage spikes during assignment seasons and drops during academic breaks, serverless architecture ensures that costs remain proportional to actual usage rather than reflecting the peak capacity of a continuously running server.

Beyond cost, serverless architecture enables faster development cycles, easier deployment pipelines, and more modular application design. Functions can be independently updated, tested, and deployed without affecting other components of the system. This modularity aligns well with microservices design principles and facilitates collaborative development in academic teams.

1.4 Motivation Behind Building ZenBot v2

The development of ZenBot v2 was motivated by the recognition that its predecessor, ZenBot AI v1, while functional as a proof-of-concept chatbot, suffered from several architectural and functional deficiencies that limited its practical utility as an educational platform. ZenBot v1 was deployed on a traditional server and featured a monolithic backend architecture in which all application logic resided in a single codebase. This design made scaling difficult, introduced single points of failure, and required manual server management for updates and maintenance.

Furthermore, ZenBot v1 lacked several critical features expected of a modern educational AI platform. There was no persistent session management, meaning users could not return to previous conversations or access their chat history. The knowledge retrieval capability was limited to a static predefined answer bank, with no ability to dynamically query external knowledge sources. PDF document processing, a highly desirable feature for students who wish to ask questions about academic papers or course materials, was entirely absent. The user authentication system was rudimentary, offering basic password checking without any backend data persistence or login history tracking.

The motivation for ZenBot v2 was thus to address these limitations comprehensively while simultaneously demonstrating mastery of cloud computing concepts through the practical

application of AWS services. The project provided an opportunity to explore serverless architecture in a real-world context, gaining hands-on experience with Lambda function design, DynamoDB schema modeling, API Gateway configuration, and S3 static hosting.

1.5 Objectives of the Project

The principal objectives of ZenBot v2 are enumerated as follows:

- To design and implement a fully serverless, cloud-native AI chatbot platform using Amazon Web Services.
- To build a responsive, user-friendly web interface hosted on Amazon S3 without reliance on traditional web servers.
- To implement a dynamic knowledge retrieval system capable of querying Wikipedia for factual information in response to user queries.
- To develop a PDF document question-answering module that allows users to upload or link PDF documents and ask relevant questions.
- To implement a session-based chat history system that persists conversations across multiple login sessions using Amazon DynamoDB.
- To provide user authentication and profile management functionality backed by DynamoDB storage.
- To demonstrate the scalability, cost efficiency, and practical effectiveness of serverless cloud architecture in an educational AI context.
- To produce a detailed academic report documenting the architecture, implementation, and analysis of the system.

1.6 Scope of the System

ZenBot v2 is scoped as an academic demonstration platform intended for use by students and educators within a university environment. The system is not intended to be a commercial product and therefore operates within the constraints of academic-grade AWS free-tier and low-cost configurations. The platform supports text-based conversational interaction, Wikipedia-sourced knowledge retrieval, and educational PDF-based question answering. It does not perform deep semantic reasoning, large language model inference, or real-time multimedia processing.

The frontend is a single-page web application accessible via a modern web browser. All backend processing is handled by AWS Lambda functions invoked through API Gateway. User data, including credentials, session information, chat messages, and PDF content chunks, is stored in Amazon DynamoDB. The system supports concurrent users in proportion to AWS Lambda's automatic scaling capabilities, and DynamoDB's provisioned or on-demand capacity settings.

1.7 Overview of the Project

ZenBot v2 is a multi-component system comprising a responsive web frontend, a serverless backend built on AWS Lambda, an API layer powered by Amazon API Gateway, and a cloud NoSQL database provided by Amazon DynamoDB. Users interact with the system through a web browser, where they can register an account, log in, initiate chat sessions, send conversational messages, query Wikipedia, upload PDF documents for question answering, and review their chat history. The entire system is deployed on AWS without any traditional server infrastructure, embodying the serverless cloud computing paradigm.

Chapter 2: Literature Review and Background

2.1 What Are Chatbots?

A chatbot is a software system designed to simulate intelligent conversation with human users, typically through a text-based interface. Chatbots can be broadly categorized into two types: rule-based systems and AI-driven systems. Rule-based chatbots operate according to a predefined set of if-then conditions, matching user input against patterns and returning scripted responses. These systems are deterministic, highly predictable, and appropriate for narrow, well-defined use cases such as FAQ automation and customer order tracking. However, they fail when users deviate from anticipated query patterns or ask novel questions outside the predefined decision tree.

AI-driven chatbots, in contrast, employ machine learning models trained on large corpora of text to understand natural language and generate contextually appropriate responses. Modern AI chatbots leverage techniques such as intent classification, named entity recognition, dialogue state tracking, and language model inference to deliver coherent, contextually aware conversation. These systems can handle significantly greater variability in user input and can be improved continuously through retraining on new data.

In recent years, the boundary between rule-based and AI-driven chatbots has blurred, with most production systems employing a hybrid approach. A base layer of deterministic rules handles common, high-confidence queries efficiently, while an AI layer manages open-ended or ambiguous requests. ZenBot v2 follows this hybrid philosophy, using predefined responses for common greetings and system commands while delegating knowledge-intensive queries to Wikipedia's API and PDF content retrieval.

2.2 Evolution of AI Assistants

The lineage of modern AI assistants can be traced through several key milestones. ELIZA, developed by Joseph Weizenbaum at MIT in 1966, is often cited as the first chatbot. ELIZA used simple pattern-matching to reflect user statements back as questions, creating an illusion of empathic understanding that some early users found remarkably convincing. ALICE (Artificial Linguistic Internet Computer Entity), developed in the 1990s using AIML (Artificial Intelligence Markup Language), introduced more sophisticated pattern matching and became the basis for numerous chatbot frameworks.

The rise of digital assistants in consumer devices marked a significant evolution. Apple's Siri, introduced in 2011, demonstrated that voice-activated AI assistants could be integrated into everyday consumer devices and used for practical tasks such as setting reminders, performing web searches, and answering factual questions. Google Assistant, Amazon Alexa, and Microsoft Cortana followed, each with increasing sophistication in natural language understanding and third-party integration capabilities. These systems benefit from cloud infrastructure that allows continuous updates to underlying models without requiring device-level software changes.

The most significant recent advancement has been the emergence of large language models (LLMs). Models in the GPT series, PaLM, Claude, and LLaMA families have demonstrated capabilities that far exceed previous AI systems in terms of language understanding, reasoning, summarization, code generation, and factual recall. These models are increasingly being deployed in educational, enterprise, and government contexts to assist with information retrieval, content generation, and decision support. While ZenBot v2 does not incorporate a full LLM for inference, its architecture is designed with future LLM integration in mind.

2.3 Serverless Computing Concepts

Serverless computing is a cloud execution model in which the cloud provider dynamically manages the allocation and provisioning of compute resources. The term 'serverless' does not imply the absence of servers but rather that the management of servers is fully abstracted away from the developer. Developers focus exclusively on writing application code in the form of functions, while the cloud provider handles infrastructure provisioning, OS-level management, scaling, fault tolerance, and billing.

The function-as-a-service (FaaS) model, which is the most common form of serverless computing, executes code in stateless, ephemeral containers that are spun up in response to triggering events and terminated after execution completes. This model imposes certain design constraints, such as statelessness between invocations and execution time limits, but these constraints encourage clean architectural design and promote the separation of concerns.

Key characteristics of serverless architectures include event-driven execution, automatic scaling, zero idle cost, and built-in high availability. AWS Lambda, Azure Functions, and Google Cloud Functions are the dominant FaaS platforms currently available. AWS Lambda supports runtimes including Python, Node.js, Java, Go, and Ruby, making it compatible with a wide variety of

application domains including web backends, data processing, machine learning inference, and IoT event processing.

2.4 Cloud Computing Architecture

Cloud computing architecture refers to the components and subcomponents required for cloud computing, including a frontend platform, a backend platform, a cloud-based delivery mechanism, and a network. Cloud architectures are generally described in terms of service models: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). Each model abstracts progressively more of the infrastructure stack from the developer or end-user.

IaaS provides virtualized computing resources such as virtual machines, storage volumes, and networking components. The developer retains full control over the operating system and application stack. PaaS extends this model by providing managed runtime environments, database services, and development frameworks, allowing developers to focus on application logic without managing the underlying infrastructure. SaaS represents the most abstracted model, delivering complete software applications over the internet that users access via web browsers or thin clients.

Modern cloud architectures for web applications typically combine multiple service models. A common pattern involves using IaaS for compute-intensive workloads that require persistent state, PaaS for managed databases and caching layers, and serverless functions for event-driven backend logic. ZenBot v2 leans heavily toward the serverless and PaaS ends of this spectrum, using S3 (object storage), Lambda (FaaS compute), API Gateway (managed API service), and DynamoDB (managed NoSQL database) to construct a complete application stack with minimal operational overhead.

2.5 Benefits of Serverless Systems

The adoption of serverless architectures offers numerous benefits over traditional server-based deployment models. Firstly, serverless removes the need for server provisioning and management, dramatically reducing the time between development completion and production deployment. For academic teams working within semester-length project cycles, this speed is of considerable practical importance.

Secondly, the auto-scaling capabilities of serverless platforms ensure that applications can handle variable workloads gracefully. As user concurrency increases, the platform automatically provisions additional function instances to handle the load without any manual intervention. Conversely, when traffic drops, the platform scales down to zero instances, eliminating idle compute costs entirely.

Thirdly, serverless architectures improve application resilience and fault tolerance. Because functions are executed in isolated containers and the platform manages instance distribution across multiple availability zones, the risk of a single point of failure causing complete application downtime is significantly reduced compared to a single-server deployment.

2.6 Educational AI Assistants

Educational AI assistants represent a growing application domain in which conversational AI is deployed to support learning outcomes. These systems range from simple FAQ bots embedded in learning management systems to sophisticated tutoring agents capable of adaptive question generation, progress tracking, and personalized feedback delivery. The potential benefits of educational AI assistants include increased access to learning support outside of formal classroom hours, reduced burden on instructors for routine information delivery, and personalized learning pathways tailored to individual student performance data.

Research in intelligent tutoring systems (ITS) has demonstrated that adaptive AI-based tutors can produce learning gains comparable to one-on-one human tutoring in certain knowledge domains, a finding known as Bloom's Two Sigma problem. However, most production educational AI systems currently deployed in universities and online learning platforms operate as informational assistants rather than true tutors, answering factual questions, summarizing content, and directing students to relevant resources rather than engaging in Socratic dialogue.

ZenBot v2 is positioned as an educational informational assistant, providing students with a tool for querying course-related PDF documents and general knowledge topics through a conversational interface. While it does not implement adaptive tutoring algorithms, its PDF QA capability represents a practical and academically relevant feature for students working with research papers, textbooks, and course notes.

2.7 Knowledge Retrieval Systems

Knowledge retrieval systems are information systems designed to surface relevant information from structured or unstructured knowledge bases in response to user queries. Traditional information retrieval relied on keyword-based indexing and Boolean query logic, as exemplified by early search engines and document management systems. While effective for exact-match searches, these systems struggled with semantic variation, synonym handling, and context-dependent relevance ranking.

Modern knowledge retrieval systems incorporate vector-based semantic search, in which documents and queries are encoded as dense embedding vectors and similarity is measured in embedding space rather than by exact keyword match. This approach, popularized by dense passage retrieval (DPR) and neural information retrieval models, enables retrieval systems to identify semantically relevant documents even when exact query terms do not appear in the target document.

ZenBot v2 implements a simplified keyword-based retrieval approach for both Wikipedia search and PDF chunk retrieval. While this approach does not match the sophistication of modern semantic search systems, it provides a functional and demonstrably effective retrieval mechanism within the constraints of an academic project. The limitations of keyword-based retrieval are acknowledged in the limitations chapter, and the migration to vector-based semantic search is proposed as a future enhancement.

2.8 Technologies Used

ZenBot v2 is built upon a technology stack that includes AWS Lambda (Python 3.9 runtime) for serverless compute, Amazon S3 for static web hosting, Amazon API Gateway for HTTP API management, Amazon DynamoDB for NoSQL data storage, PyPDF2 for PDF text extraction deployed as a Lambda Layer, the Wikipedia Python API for knowledge retrieval, and vanilla HTML/CSS/JavaScript with Bootstrap for the responsive frontend interface. The selection of these technologies reflects both their suitability for the academic use case and their widespread industry adoption, ensuring that skills developed during this project are directly transferable to professional cloud computing roles.

Chapter 3: System Methodology

3.1 Overall System Architecture

ZenBot v2 employs a fully decoupled, serverless cloud architecture in which each layer of the application stack is implemented using a distinct managed AWS service. The architecture can be conceptually divided into three tiers: the presentation tier, the logic tier, and the data tier. The presentation tier consists of the static web frontend hosted on Amazon S3. The logic tier encompasses the AWS Lambda functions that implement all business logic, accessible through Amazon API Gateway. The data tier is composed of Amazon DynamoDB tables that persist user data, chat sessions, messages, and PDF content.

This three-tier separation enforces clear boundaries between concerns and ensures that each layer can be independently scaled, maintained, and updated without affecting adjacent layers. The frontend communicates exclusively with the backend through the API Gateway HTTP endpoint, with no direct access to the database or Lambda functions. Lambda functions access DynamoDB through the AWS SDK, with permissions granted via IAM roles. This design adheres to the principle of least privilege and provides a secure, auditable access control framework.

The system is stateless at the compute layer: each Lambda invocation is independent and carries no memory of prior invocations. All state, including user authentication status, session context, and message history, is externalized to DynamoDB. This stateless compute model is a defining characteristic of serverless architecture and is critical to enabling horizontal scaling across multiple Lambda instances.

3.2 Frontend Architecture

The frontend of ZenBot v2 is a single-page web application (SPA) built using HTML5, CSS3, and vanilla JavaScript, enhanced with the Bootstrap 5 framework for responsive layout management. The decision to use vanilla JavaScript rather than a heavy JavaScript framework such as React or Angular was deliberate, motivated by the desire to minimize dependency overhead and simplify the deployment pipeline to a straightforward S3 upload.

The frontend is divided into several logical sections: a login page, a registration page, a chatbot dashboard, a session management sidebar, and a chat history display panel. Navigation between these sections is handled through JavaScript-driven view switching, with the page dynamically

showing and hiding sections without requiring full page reloads. This approach delivers a smooth user experience comparable to a framework-based SPA while maintaining simplicity in the codebase.

All API communication from the frontend is performed using the Fetch API, with requests sent asynchronously to the API Gateway endpoint. CORS headers configured on the API Gateway allow the frontend, hosted on a different domain than the backend, to make cross-origin requests without browser security errors. Response handling is implemented using JavaScript Promises and async/await syntax, ensuring that the UI remains responsive during network operations.

3.3 Backend Architecture

The backend of ZenBot v2 is implemented as a single AWS Lambda function written in Python 3.9, designed to handle multiple types of requests through a routing mechanism embedded within the function logic. Rather than deploying separate Lambda functions for each API endpoint, a unified function receives all requests and dispatches them to the appropriate internal handler based on the action parameter included in the request body. This monolithic Lambda approach was chosen for simplicity and ease of deployment during the academic project phase, with modular Lambda decomposition proposed as a future architectural improvement.

The Lambda function imports several Python libraries, including boto3 (the AWS SDK for Python), json, re, os, and wikipedia. The PyPDF2 library, which is not available in the default Lambda runtime environment, is included via a Lambda Layer that packages the dependency in a compatible format. The function is configured with an IAM execution role that grants it read and write access to the specific DynamoDB tables required for application functionality.

The Lambda function is invoked via a POST request to the API Gateway endpoint. The request body contains a JSON payload specifying the action to be performed and any required parameters such as user credentials, message content, session identifiers, or PDF URLs. The function processes the request, interacts with DynamoDB as needed, and returns a JSON response that the frontend interprets to update the UI.

3.4 API Communication Flow

The communication flow between the frontend and backend follows a standard request-response pattern mediated by Amazon API Gateway. When a user performs an action in the frontend UI,

such as sending a chat message, the JavaScript event handler constructs a JSON payload and dispatches it as an HTTP POST request to the API Gateway URL. The API Gateway, configured with CORS support and a Lambda proxy integration, forwards the request directly to the Lambda function without modification.

The Lambda function receives the event object from API Gateway, which contains the request body as a JSON-encoded string. The function parses the body, identifies the requested action, and executes the corresponding logic. After completing the action, the function constructs a response object containing an HTTP status code and a JSON-encoded result body, which API Gateway relays back to the frontend. The frontend receives the response, parses the JSON body, and updates the UI accordingly.

The entire request-response cycle typically completes within a few hundred milliseconds for standard chat operations, though PDF processing requests may take longer depending on the size of the document being processed. The API Gateway timeout is configured to accommodate extended Lambda execution times for PDF operations.

3.5 Database Structure

The data layer of ZenBot v2 consists of five Amazon DynamoDB tables, each designed to store a specific category of application data. DynamoDB's schema-less NoSQL model allows each table to store items with varying attribute sets, which provides flexibility for evolving data requirements without schema migration overhead.

The Users table stores user account information, keyed by username. Each user item contains the username as the partition key, the hashed password, the display name, email address, and any profile information the user has provided. The Sessions table stores chat session metadata, including the session identifier, the owning username, the session name, and the creation timestamp. The Messages table stores individual chat messages, with a composite key consisting of the session identifier and a timestamp-based sort key, enabling efficient retrieval of all messages within a session in chronological order.

The LoginHistory table records each successful login event, storing the username, timestamp, and IP address (if available) for audit and security purposes. The PDFChunks table stores text chunks extracted from processed PDF documents, with each item containing the source document

identifier, the chunk index, and the chunk text. During PDF question answering, relevant chunks are retrieved from this table and used to formulate the response.

3.6 Modular Design Approach

Although the Lambda function is deployed as a single unit, its internal structure follows a modular design philosophy in which each functional area of the application is handled by a dedicated Python function or code block. The main handler function acts as a router, dispatching incoming requests to sub-functions responsible for specific operations. This internal modularity reduces code coupling, improves readability, and simplifies debugging and testing.

The functional modules within the Lambda function include the authentication module, which handles login validation and user registration; the chat module, which processes conversational messages and formulates responses; the Wikipedia retrieval module, which queries the Wikipedia API; the PDF processing module, which downloads, extracts, and indexes PDF content; the session management module, which handles session creation, renaming, deletion, and history retrieval; and the profile management module, which handles user profile updates and login history queries.

Chapter 4: Implementation

4.1 AWS Services Used in This Project

ZenBot v2 is built entirely upon Amazon Web Services, utilizing a carefully selected set of services that collectively provide the compute, storage, networking, and security capabilities required for a production-grade educational AI platform. The following table summarizes the AWS services used and their roles within the system:

AWS Service	Role in ZenBot v2	Key Benefits
Amazon S3	Static website hosting for frontend	Globally accessible, cost-effective, highly available
AWS Lambda	Serverless compute for all backend logic	Auto-scaling, no server management, pay-per-use
Amazon API Gateway	HTTP API routing between frontend and Lambda	CORS support, request validation, secure access
Amazon DynamoDB	NoSQL storage for users, sessions, messages, PDFs	Millisecond latency, serverless, flexible schema
AWS IAM	Permissions and role-based access control	Secure Lambda-to-DynamoDB access enforcement
AWS Lambda Layers	PyPDF2 library packaging for PDF processing	Reusable dependencies, cleaner Lambda functions
Amazon Cognito	Authentication architecture preparation	Enterprise-grade user management for future use

Each service is described in detail in the following subsections, with particular attention to the configuration choices made and the reasoning behind them.

4.1.1 Amazon S3 — Static Website Hosting

Amazon Simple Storage Service (S3) is an object storage service that provides industry-leading scalability, data availability, security, and performance. In ZenBot v2, S3 is used for a purpose that is distinct from its primary storage function: static website hosting. S3 buckets can be configured to serve static web content (HTML, CSS, JavaScript, and image files) directly over HTTP or HTTPS, eliminating the need for a dedicated web server to serve the frontend application.

To enable static website hosting, an S3 bucket is created with a globally unique name and configured with public read access for the hosted objects. The bucket policy is set to allow s3:GetObject actions from all principals (public internet users) for the specific paths corresponding to the web application files. The bucket's static website hosting feature is enabled, with index.html specified as the index document and error.html as the error document. The S3-generated website endpoint is the URL that users access to load the ZenBot v2 frontend.

From a scalability perspective, S3's static website hosting is effectively infinitely scalable with respect to concurrent read access. S3 is designed to deliver content to millions of simultaneous users without degradation in performance, making it a highly appropriate hosting solution for a web application frontend. Furthermore, S3's pricing model, which charges only for storage used and data transferred out, makes it extremely cost-effective for serving a static frontend application with modest traffic volumes.

4.1.2 AWS Lambda — Serverless Compute

AWS Lambda is the core compute service of ZenBot v2 and the component most central to the platform's serverless architecture. Lambda allows developers to run code in response to events without provisioning or managing servers. The Lambda function for ZenBot v2 is implemented in Python 3.9 and contains the complete business logic of the backend system, organized into internal modules for routing, authentication, chat processing, knowledge retrieval, PDF handling, session management, and user profile management.

The Lambda function is configured with a memory allocation of 512 MB, which provides a balance between performance and cost for the workloads handled. More memory-intensive operations such as PDF processing benefit from the additional compute power that higher memory settings provide, as Lambda allocates CPU proportionally to the memory setting. The function timeout is set to 60 seconds, which is sufficient for all standard operations including Wikipedia queries and moderately sized PDF processing tasks.

Lambda's invocation model in ZenBot v2 is synchronous, triggered by HTTP requests forwarded from API Gateway. Each API request results in a single Lambda invocation, which processes the request, interacts with DynamoDB as required, and returns the response. Lambda's cold start behavior, in which the first invocation after a period of inactivity incurs additional latency while the function container initializes, is acceptable for the academic use case of this platform. For

production systems requiring guaranteed low-latency responses, provisioned concurrency would be employed to eliminate cold starts.

The Lambda function uses the boto3 library to interact with DynamoDB, calling put_item, get_item, query, and scan operations as appropriate for each action type. The wikipedia Python library is used for knowledge retrieval, wrapping the Wikipedia API in a convenient Python interface. Error handling within the Lambda function follows a try-except pattern, with errors logged to AWS CloudWatch Logs and appropriate error responses returned to the frontend.

4.1.3 Amazon API Gateway — HTTP API

Amazon API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale. In ZenBot v2, API Gateway serves as the single entry point for all backend API requests, routing HTTP POST requests from the frontend to the Lambda function. The API is configured as an HTTP API (as distinct from the REST API type), which offers lower latency, lower cost, and simpler configuration for the use case at hand.

A single POST route is configured on the API Gateway, mapped to the Lambda integration. All frontend requests, regardless of the action being requested, are sent to this single route with different action parameters in the JSON body. The Lambda function performs internal routing based on the action field, as described in the backend architecture section. This design simplifies the API Gateway configuration while keeping all routing logic within the Lambda codebase, where it can be easily modified and tested.

Cross-Origin Resource Sharing (CORS) configuration is a critical aspect of the API Gateway setup. Because the frontend is served from an S3 domain and the API is hosted on a different API Gateway domain, browsers enforce CORS restrictions that would otherwise block frontend-to-API requests. The API Gateway is configured to include appropriate CORS headers in its responses, specifying the allowed origin, allowed methods, and allowed headers. This configuration enables the browser to permit cross-origin POST requests from the S3-hosted frontend to the API Gateway endpoint.

4.1.4 Amazon DynamoDB — NoSQL Database

Amazon DynamoDB is a fully managed, serverless, key-value and document database designed for high-performance applications at any scale. DynamoDB delivers single-digit millisecond performance at any scale, making it well-suited for the real-time query requirements of a

conversational AI platform. In ZenBot v2, DynamoDB serves as the persistent storage layer for all user data, chat data, and PDF content.

The five DynamoDB tables used in the system are designed with access patterns as the primary design consideration, in accordance with DynamoDB's single-table or multi-table design philosophies. The Users table uses username as the partition key, enabling O(1) lookups by username during login validation. The Sessions table uses a session_id partition key (a UUID generated at session creation time) with a secondary index on username to support efficient retrieval of all sessions belonging to a specific user. The Messages table uses session_id as the partition key and timestamp as the sort key, enabling range queries to retrieve all messages in a session ordered by time.

DynamoDB's pay-per-request pricing mode (on-demand capacity) is used for all tables, meaning the system incurs no database costs during periods of inactivity and scales automatically to accommodate any request volume. This pricing model is ideal for the variable traffic patterns of an academic platform and eliminates the need to estimate and provision read and write capacity units in advance.

4.1.5 AWS IAM — Permission Management

AWS Identity and Access Management (IAM) is the access control service used to manage permissions within the ZenBot v2 architecture. IAM enables fine-grained control over which AWS services and resources can be accessed by which entities, using policies that define allowed and denied actions on specific resources. In the context of ZenBot v2, IAM is primarily used to define the execution role assigned to the Lambda function, controlling which DynamoDB operations the function is permitted to perform.

The Lambda execution role is configured with a policy that grants the lambda function permission to perform dynamodb:PutItem, dynamodb:GetItem, dynamodb:Query, dynamodb:Scan, dynamodb:UpdateItem, and dynamodb:DeleteItem operations on the specific DynamoDB tables used by the application. No permissions are granted for other AWS services or for DynamoDB operations not required by the application, adhering to the principle of least privilege. This approach minimizes the potential damage from any security compromise of the Lambda function, as the function cannot perform unauthorized operations even if its code is modified by a malicious actor.

The S3 bucket used for frontend hosting is configured with a separate bucket policy that allows public GetObject access to the web application files. API Gateway does not require explicit IAM configuration for the Lambda proxy integration, as permissions are managed through a resource-based policy on the Lambda function that allows API Gateway to invoke it.

4.1.6 AWS Lambda Layers — Dependency Management

AWS Lambda Layers are a distribution mechanism for libraries, custom runtimes, or other function dependencies. A Lambda Layer is a ZIP archive that contains libraries, a custom runtime, or other dependencies, which can be attached to one or more Lambda functions. When a Layer is attached to a function, the Layer's contents are extracted to the /opt directory of the function execution environment, making the layer's libraries available for import in the function code.

In ZenBot v2, a Lambda Layer is used to include the PyPDF2 library in the function's execution environment. PyPDF2 is a Python library for reading and extracting text from PDF files, which is not included in the default AWS Lambda Python runtime. To create the Layer, PyPDF2 is installed into a local Python packages directory using pip, the directory is archived into a ZIP file with the required directory structure (python/lib/python3.9/site-packages/), and the ZIP file is uploaded to AWS and registered as a Lambda Layer. The Layer is then attached to the Lambda function, making PyPDF2 available for use in the PDF processing module.

Using Lambda Layers for dependency management offers several advantages over including dependencies directly in the function deployment package. Layers can be shared across multiple Lambda functions, reducing code duplication. They can be versioned independently of the function code, allowing dependency updates without function redeployment. They also reduce the size of the function deployment package, potentially improving cold start times by reducing the amount of code that must be loaded into the execution environment.

4.1.7 Amazon Cognito — Authentication Architecture

Amazon Cognito is a fully managed user authentication, authorization, and user management service for web and mobile applications. Cognito supports user registration, login, multi-factor authentication, social identity provider federation, and fine-grained access control through identity pools. In ZenBot v2, Cognito is included as part of the system architecture as a planned enhancement for the authentication subsystem.

Currently, the authentication system relies on DynamoDB-stored credentials with basic validation in the Lambda function. While this implementation is functional for the academic scope of the project, it does not meet production security standards. The architecture has been designed to accommodate future integration of Cognito as the authentication provider, which would provide industry-standard security features including secure password hashing through Cognito's built-in algorithms, email verification, password recovery workflows, and JWT-based session tokens.

Implementing Cognito as part of this project served an important educational purpose: understanding the role of managed authentication services in cloud application architecture and the considerations involved in designing a system that can evolve from a simple credential-based authentication model to a fully managed enterprise-grade identity solution without requiring fundamental architectural changes.

4.2 Frontend Interface Implementation

4.2.1 Chatbot UI

The chatbot user interface is the primary point of interaction between the user and the ZenBot v2 system. It is designed to be clean, intuitive, and responsive, providing a chat experience consistent with modern messaging applications. The main chat area displays a scrollable message feed in which user messages appear right-aligned with a distinct background color, while bot responses appear left-aligned with a contrasting color scheme. Each message is timestamped and includes the sender's identifier.

The message input area at the bottom of the chat interface consists of a text input field and a send button. Users can type their messages and submit them by pressing Enter or clicking the send button. The input field is cleared after message submission, and the chat feed automatically scrolls to display the most recent messages. A visual loading indicator is displayed while the backend processes the request, providing the user with feedback that the system is responding.

Special UI elements are included for the PDF question-answering workflow. A dedicated input field allows users to paste a Google Drive share link or direct URL to a PDF document. When a PDF URL is submitted, the system processes the document in the background and displays a confirmation message once the PDF has been indexed. Subsequently, users can ask questions about the document's content in the main chat interface.

4.2.2 Login System

The login system presents users with a clean authentication form requesting a username and password. Upon form submission, the credentials are sent to the API Gateway endpoint with an action of 'login'. The Lambda function validates the credentials against the Users DynamoDB table, returning either a success response containing the user's display name and profile information or an error response with an appropriate message. Successful login transitions the UI to the chatbot dashboard, while failed login attempts display an error notification without clearing the form fields.

The registration form collects a username, password, display name, and email address. Usernames are checked for uniqueness against the Users table before account creation, and appropriate feedback is provided if the requested username is already taken. Password confirmation is validated client-side before the registration request is sent to the backend, reducing unnecessary API calls for easily detectable input errors.

4.2.3 Session Management

Session management is exposed through a sidebar panel on the chatbot dashboard. The sidebar displays a list of the user's existing chat sessions, each represented by its session name and creation date. Clicking a session in the sidebar loads the full message history of that session in the main chat area. Each session entry has options to rename or delete the session, accessible through a context menu or inline action buttons.

New sessions can be created by clicking the 'New Chat' button, which sends a session creation request to the backend, generates a new UUID for the session, and adds it to the session list in the sidebar. The new session is immediately selected as the active session, and the chat area is cleared to receive the first message of the new conversation.

4.3 Backend Logic Implementation

The Lambda function's main handler receives the API Gateway proxy event, parses the JSON request body, and extracts the action field to determine which internal handler to invoke. The routing logic is implemented as a series of conditional checks that map action strings to handler functions. This approach is analogous to a URL router in a traditional web framework, adapted for the single-function serverless deployment model.

Each handler function is responsible for performing the required DynamoDB operations, calling any external APIs (such as the Wikipedia API or HTTP requests for PDF downloads), constructing the response payload, and returning it to the main handler. Error handling is implemented at both the individual handler level and the main handler level, ensuring that unhandled exceptions are caught and returned as structured error responses rather than causing the Lambda function to crash with an unformatted error.

4.4 Wikipedia Knowledge Retrieval Module

The Wikipedia knowledge retrieval module is invoked when the Lambda function's message classifier determines that a user's message is a knowledge query rather than a conversational exchange. The classification is performed using keyword-based heuristics, checking for question words (what, who, where, when, why, how) and the presence of topic-like noun phrases in the user's input.

When a knowledge query is detected, the module calls the wikipedia.search() function to retrieve a list of potentially relevant Wikipedia page titles matching the user's query. The top result is selected, and wikipedia.summary() is called to retrieve the introductory summary of the corresponding Wikipedia article. The summary is truncated to a configurable maximum length and returned as the bot's response, providing the user with a concise but informative answer to their factual query.

Error handling in the Wikipedia module addresses cases such as disambiguation pages (where the search term matches multiple Wikipedia articles), page not found errors, and network timeouts. In these cases, appropriate fallback messages are returned to the user, and the exception is logged to CloudWatch for debugging purposes.

4.5 PDF Processing Module

The PDF processing module is one of the most complex and technically significant components of ZenBot v2. It implements a complete pipeline from PDF acquisition through text extraction, chunking, indexing, and retrieval. The module is designed to handle Google Drive share links in addition to direct PDF URLs, which is a common scenario in academic settings where students share documents via Google Drive.

When a PDF URL is received, the module first checks whether it is a Google Drive share link and, if so, transforms it into a direct download URL by modifying the URL path to use the export=download format. A standard HTTP GET request is then issued to download the PDF file content into memory as a bytes stream. This approach avoids writing the PDF to disk, which is not reliably supported in the Lambda execution environment, and instead processes the content entirely in memory.

The PyPDF2 library is used to parse the in-memory PDF bytes and extract the text content page by page. Pages are concatenated into a single text string, which is then split into overlapping chunks of a configurable maximum size (typically 500 words per chunk with a 50-word overlap). Overlapping chunks are used to ensure that answers that span paragraph boundaries are not lost due to chunk boundaries.

Each text chunk is stored in the PDFChunks DynamoDB table along with the source document URL and chunk index. When a user asks a question about a previously processed PDF, the module retrieves all chunks for that document from DynamoDB and performs a keyword-based relevance search to identify the chunk most likely to contain the answer. The relevance search counts the number of words from the user's question that appear in each chunk and returns the chunk with the highest overlap count as the response.

4.6 Chat Session System

The session management module handles all operations related to the lifecycle of chat sessions. Sessions are identified by UUIDs generated at creation time, ensuring global uniqueness without requiring a centralized counter or sequence generator. The creation timestamp is recorded in the Sessions table, and the default session name is set to a formatted string representing the creation time, which users can subsequently rename.

Loading a session's message history involves a DynamoDB query on the Messages table using the session_id as the partition key. The results are sorted by the timestamp sort key in ascending order to reconstruct the chronological conversation. The frontend renders each message with its sender identity and timestamp, recreating the original conversation context for the user.

Session deletion is implemented as a cascading operation that removes both the session record from the Sessions table and all associated messages from the Messages table. This cascade is

implemented in the Lambda function rather than relying on DynamoDB's limited cascade features, involving a query to retrieve all message items for the session followed by batch deletion operations.

4.7 User Management System

The user management system handles registration, login, profile updates, and login history. During registration, the provided username is checked for uniqueness in the Users table using a get_item operation. If the username is not found, a new user item is created with the provided credentials and profile information. Passwords are stored with basic encoding in the academic version of the system, with full cryptographic hashing proposed as a security enhancement for future iterations.

Login validation involves retrieving the user item from the Users table by username and comparing the provided password against the stored value. Upon successful authentication, a login history record is written to the LoginHistory table capturing the username, login timestamp, and any available session metadata. The frontend stores the authenticated username in JavaScript session state to include it in subsequent API requests that require user identification.

Profile updates allow users to change their display name and email address. The Lambda function performs a DynamoDB update_item operation to modify the corresponding attributes of the user's record in the Users table. The updated profile information is returned in the response and reflected in the frontend UI.

Chapter 5: System Workflow

5.1 Complete User Journey Walkthrough

This chapter presents a detailed step-by-step description of the complete workflow of the ZenBot v2 system, from the initial user access through all major functional paths. Understanding this end-to-end flow is essential for appreciating how the individual components described in the architecture and implementation chapters work together to deliver a coherent user experience.

5.2 Application Access and Frontend Loading

A user accesses ZenBot v2 by navigating to the S3 static website URL in their web browser. The browser sends an HTTP GET request to the S3 endpoint, which returns the index.html file along with its associated CSS and JavaScript assets. The browser parses the HTML, loads the stylesheets and scripts, and renders the login page as the initial view of the application. The entire frontend loading process is handled by S3 without any backend compute involvement, demonstrating the efficiency of static website hosting for web application frontends.

5.3 Authentication Flow

Upon viewing the login page, the user enters their username and password and submits the form. The JavaScript event handler prevents the default form submission behavior and instead constructs a JSON payload containing the action 'login', the username, and the password. This payload is sent as a POST request to the API Gateway URL. API Gateway receives the request, validates it against the configured HTTP route, and forwards it to the Lambda function.

The Lambda function parses the request body, identifies the 'login' action, and invokes the authentication handler. The handler queries the Users DynamoDB table for the item with the provided username key. If the item is found and the stored password matches the provided password, the handler constructs a success response containing the user's display name and profile information. The response travels back through API Gateway to the frontend, where the JavaScript handler processes it, stores the username in session state, and transitions the view to the chatbot dashboard.

5.4 Chat Interaction Workflow

Once on the chatbot dashboard, the user can initiate a new session or select an existing session from the sidebar. If creating a new session, the frontend sends a 'create_session' action to the backend, which generates a UUID, creates a record in the Sessions DynamoDB table, and returns the new session ID to the frontend. The session is added to the sidebar and selected as the active session.

When the user types a message and submits it, the frontend sends a 'send_message' action along with the message text, session ID, and username. The Lambda function receives the request and first stores the user's message in the Messages DynamoDB table with the current timestamp. The function then analyzes the message content to determine the appropriate response strategy.

If the message matches a predefined response pattern (such as a greeting or help request), the predefined response is returned directly. If the message is identified as a knowledge query based on its structure and vocabulary, the Wikipedia retrieval module is invoked to search for relevant information. The Wikipedia summary is returned as the bot response. If the message is a PDF-related question and a PDF has been previously processed for the session, the PDF chunk retrieval module is invoked to find the most relevant text chunk. In all cases, the bot response is stored in the Messages DynamoDB table alongside the original user message.

5.5 PDF Question Answering Workflow

The PDF question answering workflow begins when the user pastes a PDF URL into the designated input field and submits it. The frontend sends a 'process_pdf' action with the URL to the backend. The Lambda function receives the request and invokes the PDF processing pipeline: the URL is normalized if it is a Google Drive link, the PDF is downloaded into memory, PyPDF2 extracts the text, the text is split into chunks, and the chunks are stored in the PDFChunks DynamoDB table.

Upon successful processing, the Lambda function returns a confirmation message indicating the number of pages and chunks extracted. The frontend displays this confirmation in the chat interface, informing the user that the PDF is ready for questions. Subsequent messages from the user that contain question keywords trigger the PDF retrieval path, in which the Lambda function computes keyword overlap scores for all stored chunks and returns the most relevant chunk as the response.

5.6 Session History Retrieval

When a user clicks a session in the sidebar, the frontend sends a 'get_messages' action with the session ID to the backend. The Lambda function queries the Messages DynamoDB table for all items with the given session ID, sorted by timestamp. The messages are returned to the frontend as a JSON array, which the JavaScript handler uses to render the complete conversation history in the chat area. This allows users to review previous conversations and continue them seamlessly across different login sessions.

Chapter 6: ZenBot v1 vs ZenBot v2 — Comparative Analysis

6.1 Introduction to the Comparison

ZenBot AI v1 was the first iteration of the conversational AI platform developed as a precursor to ZenBot v2. While v1 established the foundational concept of a cloud-deployed chatbot for educational use, it suffered from significant architectural, functional, and user experience limitations that constrained its practical value. ZenBot v2 was developed specifically to address these limitations through a comprehensive redesign of the system architecture and a substantial expansion of the feature set.

The following comparison covers eight dimensions of the system: interface quality, cloud architecture, features, knowledge capability, educational usefulness, scalability, user experience, and backend design. A summary comparison table is presented, followed by detailed discussion of the improvements made in each dimension.

Aspect	ZenBot AI v1	ZenBot v2
User Interface	Basic HTML/CSS with minimal design	Responsive Bootstrap-based UI with session sidebar
Cloud Architecture	Monolithic server deployment	Fully serverless AWS architecture
Hosting	Traditional web server or local	Amazon S3 static website hosting
Backend	Single script server-side logic	Modular AWS Lambda functions
API Layer	Direct HTTP requests, no gateway	Amazon API Gateway with CORS and routing
Database	Local storage or flat file system	Amazon DynamoDB NoSQL database
Authentication	Simple password check	DynamoDB-backed login with profile management
Knowledge Retrieval	Predefined answer bank only	Wikipedia API + contextual keyword search
PDF Support	Not available	Full PDF download, extraction, chunking, and QA
Chat Sessions	No session tracking	Full session create/rename/delete/history system
Login History	Not tracked	Login timestamps stored in DynamoDB

Aspect	ZenBot AI v1	ZenBot v2
Scalability	Limited to server capacity	Auto-scales with AWS Lambda
Cost Model	Fixed server cost	Pay-per-use serverless pricing
Educational Use	Basic topic answers	PDF-based document QA and Wikipedia knowledge
Deployment	Manual server setup	Cloud-native S3 + Lambda + API Gateway

6.2 Interface Quality

ZenBot AI v1 featured a minimal HTML/CSS interface with no responsive design, making it difficult to use on mobile devices or smaller screen sizes. The color scheme was utilitarian, with no visual hierarchy between user messages and bot responses. Navigation between different sections of the application required full page reloads, disrupting the conversational flow.

ZenBot v2 introduces a professionally designed interface built with Bootstrap 5, featuring a responsive layout that adapts gracefully to different screen sizes. The session sidebar, chat area, and input controls are logically organized, providing users with an intuitive mental model of the application's structure. User and bot messages are visually differentiated through contrasting colors and alignment, and smooth CSS transitions enhance the perceived performance of the application.

6.3 Cloud Architecture

The most fundamental difference between v1 and v2 lies in the cloud architecture. ZenBot AI v1 was deployed on a traditional server instance (or run locally), requiring continuous provisioning of compute resources regardless of actual usage. This approach incurred fixed costs, required manual server management, and introduced a single point of failure. Server downtime during maintenance windows resulted in complete application unavailability.

ZenBot v2 adopts a fully serverless architecture built on AWS services. The compute layer (Lambda) runs only when invoked and scales automatically to accommodate concurrent requests. The frontend is served from S3 with globally distributed availability. The database (DynamoDB) is a fully managed serverless service with automatic scaling and built-in replication. This architecture eliminates single points of failure, reduces operational overhead to near zero, and ensures that the system remains available and performant under varying load conditions.

6.4 Features

ZenBot AI v1 offered a limited feature set centered on basic conversational response. The predefined answer bank supported a finite set of topics, and there was no mechanism for dynamically expanding the knowledge base. There was no PDF processing capability, no session-based chat history, no login history tracking, and no profile management beyond basic authentication.

ZenBot v2 introduces a comprehensive feature set that addresses all of these gaps. Dynamic Wikipedia knowledge retrieval eliminates the reliance on a static answer bank for factual queries. The PDF processing pipeline enables users to query the contents of any PDF document accessible via URL. Session-based chat history provides persistent conversation memory. Profile management allows users to maintain their account information. Login history tracking provides an audit trail of authentication events.

6.5 Knowledge Capability

The knowledge capability of ZenBot AI v1 was strictly limited to the predefined response bank, which covered a small number of topics with fixed responses. Any query outside this set resulted in a generic fallback response. This limitation severely constrained the system's practical utility as an information tool.

ZenBot v2's Wikipedia integration dramatically expands the knowledge capability by providing access to the entirety of Wikipedia's content through dynamic API queries. The system can respond to questions about virtually any topic covered by Wikipedia, including historical events, scientific concepts, biographical information, geographical facts, and cultural topics. The addition of PDF-based QA further extends the system's knowledge reach to user-provided documents, which is particularly valuable in academic settings where course-specific materials may not be covered by general knowledge sources.

6.6 Scalability

ZenBot AI v1's server-based deployment model provided scalability only through vertical scaling (upgrading the server instance to a more powerful configuration) or horizontal scaling (adding more server instances behind a load balancer), both of which required manual intervention and incurred additional infrastructure costs. In practice, the academic deployment did not implement

any formal scaling strategy, meaning the system's capacity was fixed at the capacity of a single server.

ZenBot v2's serverless architecture provides automatic horizontal scaling as an inherent property of the platform. AWS Lambda can execute thousands of concurrent function invocations simultaneously without any configuration changes, with each invocation running in an independent container. DynamoDB's on-demand capacity mode similarly scales read and write throughput automatically in response to traffic patterns. This architectural advantage ensures that ZenBot v2 can accommodate any realistic growth in user base without requiring infrastructure changes.

Chapter 7: Advantages of the System

7.1 Scalable Architecture

One of the most significant advantages of ZenBot v2 is its inherently scalable architecture. By leveraging AWS Lambda for compute and DynamoDB for storage, the system scales automatically in response to user demand without requiring manual intervention or infrastructure provisioning. Lambda's concurrency model allows the system to handle sudden spikes in user traffic gracefully, spinning up additional function instances as needed and returning to a minimal footprint during periods of low activity. This elastic scalability ensures a consistent user experience regardless of the number of simultaneous users accessing the platform.

The scalability extends to the data tier as well. DynamoDB's on-demand capacity mode eliminates the need to predict and pre-provision read and write capacity, automatically adjusting to the actual workload. As the volume of stored chat sessions, messages, and PDF chunks grows with user adoption, DynamoDB handles the increased storage and query load without any database administration effort. This combination of compute and storage scalability makes ZenBot v2 suitable for growth from a small academic cohort to a larger institutional deployment without architectural changes.

7.2 Serverless Cost Efficiency

The pay-per-use pricing model of AWS Lambda and DynamoDB makes ZenBot v2 highly cost-efficient, particularly for the variable traffic patterns typical of an academic platform. Lambda charges are based on the number of invocations and the execution duration measured in gigabyte-seconds, meaning no charges are incurred during periods when the system is not in use. For a platform that experiences high usage during the academic semester and minimal usage during holidays and exam study periods, this pricing model can result in substantially lower costs compared to a fixed-cost server deployment.

S3 pricing for static website hosting is also highly favorable, with costs based on the storage volume occupied by the web application files (typically a few megabytes) and the data transferred in response to user requests. For an academic platform with modest traffic, S3 hosting costs are negligible. The combination of Lambda, DynamoDB, S3, and API Gateway costs can remain

within the AWS free tier for a lightly used academic deployment, making ZenBot v2 essentially free to operate for small-scale use.

7.3 Modular Design

The modular internal design of the Lambda function facilitates maintainability, testability, and future enhancement. Each functional module within the Lambda function is self-contained and operates independently of other modules, with well-defined interfaces for input and output. This modularity allows individual modules to be tested and debugged in isolation, and new modules can be added without modifying existing ones.

The modular design also provides a clear migration path toward a fully decomposed microservices architecture, in which each module would be deployed as a separate Lambda function with its own API Gateway route. This migration could be undertaken incrementally, module by module, as the system's scale and complexity grow to justify the additional operational complexity of managing multiple function deployments.

7.4 Educational Usefulness

ZenBot v2 is specifically designed with educational use cases in mind, and its feature set reflects the information needs of university students. The Wikipedia integration provides instant access to encyclopedic knowledge on virtually any academic topic, reducing the friction of switching between a chat interface and a web browser for factual lookups. The PDF question-answering capability is particularly valuable in educational contexts, where students frequently need to extract specific information from lengthy research papers, textbooks, and lecture notes.

The session-based chat history feature supports extended learning workflows in which students build up a body of knowledge through successive conversations with the system. By preserving previous conversations, the system allows students to review earlier explanations, track their learning progress, and return to unresolved questions at a later time. These features collectively make ZenBot v2 a practical tool for academic research, study assistance, and information retrieval.

7.5 Cloud-Native Deployment

ZenBot v2 is designed from the ground up as a cloud-native application, meaning it does not merely run on the cloud but is architected to exploit the capabilities of cloud infrastructure at every

level. The use of managed services (S3, Lambda, API Gateway, DynamoDB) rather than self-managed applications running on virtual machines means that the system benefits from the operational reliability, security patching, automatic scaling, and continuous improvement that AWS applies to these services. Cloud-native deployment also means that the system is inherently multi-region capable, with relatively modest configuration changes required to deploy replicas of the system in multiple AWS regions for reduced latency and increased fault tolerance.

7.6 Improved User Experience

Compared to ZenBot AI v1, the user experience of ZenBot v2 represents a substantial improvement across all dimensions. The responsive Bootstrap-based interface provides a professional and aesthetically coherent presentation on both desktop and mobile devices. The session sidebar provides clear navigation between concurrent and historical conversations. Real-time feedback during message processing keeps users informed about the system's status. The PDF processing confirmation message provides actionable feedback about the document indexing process.

Chapter 8: Limitations of the System

8.1 Limited AI Reasoning Capability

ZenBot v2 does not incorporate a large language model or any deep learning-based natural language processing component. The chatbot's conversational capability is limited to predefined responses for scripted interactions and Wikipedia summaries for factual queries. It cannot engage in multi-turn reasoning, follow complex chains of logic, maintain contextual awareness across multiple message exchanges, or generate novel responses beyond what is available in its predefined answer bank and Wikipedia content. Users interacting with the system will encounter the boundaries of this limitation when asking nuanced, ambiguous, or context-dependent questions that require inferential reasoning.

8.2 Simple Keyword Matching for PDF QA

The PDF question-answering module employs simple keyword overlap counting to identify the most relevant text chunk for a given user question. This approach, while functional for straightforward factual questions where the answer text shares vocabulary with the question, fails for semantically complex queries where the answer may be expressed using different vocabulary than the question. For example, a question about the 'impact of a factor' may fail to retrieve a chunk that discusses the 'effect of the same factor' using synonymous terms. This limitation is inherent to keyword-based retrieval and can only be addressed through the adoption of semantic vector search techniques, as discussed in the future improvements chapter.

8.3 Dependence on Wikipedia for External Knowledge

The system's external knowledge retrieval capability is entirely dependent on the Wikipedia API. Wikipedia, while an extraordinarily broad and generally reliable reference source, has several limitations in an academic context. It may not contain information about highly specialized academic topics, recent publications, proprietary domain knowledge, or institution-specific information. Furthermore, Wikipedia articles vary in quality and coverage, and the summary extracted by the API may not always represent the most relevant or accurate portion of the article for the user's specific query.

8.4 Basic Authentication Security

The current authentication implementation stores user credentials in DynamoDB with minimal security protections. In a production system, passwords would be hashed using a cryptographic algorithm such as bcrypt or Argon2 before storage, ensuring that even if the database contents were compromised, the original passwords could not be recovered. The current implementation does not implement this level of security, making it unsuitable for deployment in a context where real user credentials are at risk. This limitation is acknowledged and the integration of Amazon Cognito or a password hashing library is proposed as an immediate security enhancement.

8.5 No Real-Time Communication

The current architecture uses a request-response HTTP model in which the frontend polls the backend for responses. This model introduces latency in message delivery and does not support real-time features such as streaming responses, typing indicators, or live session sharing. For a more responsive conversational experience, a WebSocket-based communication model would be more appropriate, potentially implemented using AWS API Gateway's WebSocket API support.

8.6 Lambda Cold Start Latency

AWS Lambda's cold start behavior introduces variable latency in response to the first request received after a period of inactivity. Cold starts occur when Lambda must initialize a new function container, which involves loading the function code and its dependencies (including the PyPDF2 Lambda Layer) into memory. For the ZenBot v2 function, cold start times may range from a few hundred milliseconds to several seconds, depending on the function memory allocation and the size of the deployment package. While acceptable for the academic use case, cold start latency would need to be addressed through provisioned concurrency or architectural changes for a production deployment requiring guaranteed response times.

Chapter 9: Future Improvements

9.1 Vector Search and Semantic Retrieval

The most impactful technical improvement that could be made to ZenBot v2's PDF QA capability is the replacement of keyword-based chunk retrieval with vector-based semantic search. In a vector search system, both the text chunks stored in the database and the user's query are encoded as high-dimensional embedding vectors using a pre-trained sentence embedding model. Similarity between the query and each chunk is then computed as the cosine distance between their respective embedding vectors, identifying the most semantically similar chunk regardless of vocabulary overlap.

This approach would dramatically improve the relevance of retrieved answers for semantically complex queries. Vector databases such as Amazon OpenSearch Service (with k-NN search), Pinecone, or Weaviate could replace or augment the DynamoDB-based chunk storage to provide efficient approximate nearest-neighbor search over large embedding collections. The integration of a sentence transformer model for embedding generation could be achieved either through a Lambda function using the sentence-transformers Python library or through Amazon SageMaker inference endpoints.

9.2 Retrieval-Augmented Generation (RAG) Architecture

Retrieval-Augmented Generation (RAG) is an architectural pattern in which a document retrieval system is combined with a large language model to produce answers that are grounded in specific retrieved documents rather than relying solely on the LLM's parametric knowledge. In a RAG pipeline, the user's query is used to retrieve the most relevant document chunks, which are then provided to the LLM as context along with the query. The LLM generates a natural language answer that synthesizes information from the retrieved context.

Implementing RAG in ZenBot v2 would transform the PDF QA module from a simple text retrieval system into a genuine question-answering system capable of generating synthesized, coherent answers from complex multi-paragraph document context. The LLM component could be provided through the Amazon Bedrock service, which offers access to multiple foundation models including Anthropic Claude, Amazon Titan, and Meta Llama through a unified API.

Integration with Bedrock would be straightforward given that both services are within the AWS ecosystem and can be accessed using the boto3 SDK from Lambda.

9.3 Real LLM Integration

Beyond the RAG use case, integrating a large language model directly into ZenBot v2's conversational pipeline would substantially improve the quality of all chatbot interactions, not just PDF QA. An LLM would enable the system to engage in multi-turn contextual dialogue, generate nuanced and synthesized responses to complex queries, handle ambiguous or poorly structured questions gracefully, and provide explanations, summaries, and analyses that go beyond simple factual retrieval.

Amazon Bedrock provides the most natural integration path, offering serverless LLM inference through an API that is compatible with the Lambda-based backend architecture of ZenBot v2. Alternatively, OpenAI's API or Anthropic's Claude API could be called from within the Lambda function to provide LLM inference capabilities. The primary consideration in selecting an LLM provider would be cost, as LLM inference is significantly more expensive per request than the current Wikipedia and keyword search operations.

9.4 Improved Authentication with Amazon Cognito

As discussed in the limitations chapter, the current authentication implementation does not meet production security standards. Replacing the DynamoDB-based credential storage with Amazon Cognito would provide enterprise-grade authentication security including bcrypt-based password hashing, email verification, password recovery, multi-factor authentication, and JWT-based session management. Cognito integrates natively with API Gateway and Lambda, enabling fine-grained access control based on user identity and group membership. This integration would also enable social login through OAuth providers such as Google and Facebook, reducing friction for new user registration.

9.5 Mobile-First Responsive Design

While the current Bootstrap-based frontend provides basic responsive design, a dedicated mobile-first redesign using a modern frontend framework such as React Native (for native mobile apps) or a Progressive Web App (PWA) approach would significantly improve the mobile user experience. PWAs offer native app-like features including offline access, push notifications, and

installation on the device home screen, all within the web browser environment. Converting ZenBot v2 to a PWA would require adding a service worker for offline caching and a web app manifest, both of which can be added to the existing static frontend without backend changes.

9.6 Advanced Session Context Management

A meaningful enhancement to the conversational capability of ZenBot v2 would be the incorporation of session-level context awareness in the Lambda function's response generation. Currently, each message is processed independently without reference to the conversation history. A context-aware system would include the last N messages of the session as context when generating a response, enabling the bot to maintain topic continuity, refer back to earlier statements, and provide more coherent multi-turn dialogue. This enhancement could be implemented using a windowed context approach within the Lambda function, retrieving the most recent messages from DynamoDB and including them in the LLM prompt or knowledge retrieval query.

9.7 Analytics and Usage Dashboard

An administrative analytics dashboard would provide valuable insights into platform usage patterns, popular query topics, PDF processing volumes, and user engagement metrics. AWS services such as Amazon CloudWatch, AWS Athena, and Amazon QuickSight could be used to collect, store, query, and visualize usage data. CloudWatch already collects Lambda invocation metrics, duration, and error rates, providing a starting point for operational monitoring. Custom metrics published from the Lambda function could track application-level events such as Wikipedia query frequency, PDF processing counts, and session creation rates, enabling data-driven decisions about feature development priorities.

Chapter 10: Conclusion

10.1 Summary of Achievements

ZenBot v2 has been successfully designed, implemented, and deployed as a fully functional serverless AI knowledge platform on Amazon Web Services. The system demonstrates the practical application of core cloud computing principles in the context of an educational AI tool, delivering a feature set that substantially exceeds the capabilities of its predecessor, ZenBot AI v1, while operating within a modern, scalable, and cost-efficient cloud architecture.

The implemented system provides users with a responsive web interface hosted on Amazon S3, serverless backend processing through AWS Lambda, HTTP API management via Amazon API Gateway, persistent data storage in Amazon DynamoDB, dynamic knowledge retrieval from Wikipedia, PDF document question-answering capability using PyPDF2 and keyword-based retrieval, session-based chat history management, user authentication and profile management, and login history tracking. Each of these features is fully operational and has been validated through functional testing.

10.2 Demonstration of Serverless Cloud Architecture

Perhaps the most significant outcome of this project is the demonstration that a fully featured educational AI platform can be built and deployed using exclusively serverless cloud services, without any traditional server infrastructure. The architecture of ZenBot v2 serves as a concrete illustration of how serverless paradigms translate from theoretical concepts to practical implementations: Lambda provides elastic, event-driven compute; DynamoDB provides schema-flexible, auto-scaling persistent storage; API Gateway provides managed, CORS-enabled API routing; and S3 provides globally available, infinitely scalable static content delivery.

The system operates at near-zero cost during periods of inactivity and scales automatically to accommodate any realistic level of concurrent usage. The operational overhead is minimal: there are no servers to patch, no capacity to manage, and no infrastructure failures to handle manually. This combination of features makes the serverless architecture not merely an academic exercise but a genuinely practical and superior alternative to traditional server-based deployment for the class of application that ZenBot v2 represents.

10.3 Learning Outcomes

The development of ZenBot v2 provided comprehensive hands-on experience with Amazon Web Services across multiple service categories. The project required practical application of knowledge in serverless function design, NoSQL database modeling, API design and deployment, static web hosting, IAM security policy configuration, Lambda Layer construction, and cross-origin web application architecture. These skills are directly relevant to cloud computing professional roles and represent a significant practical supplement to theoretical course instruction.

Beyond the technical skills, the project also developed competencies in system design, architectural decision-making, technical documentation, and the evaluation of design trade-offs. The comparative analysis between v1 and v2 required clear articulation of the architectural and functional improvements, reinforcing the ability to reason about system design choices in the context of real requirements.

10.4 Importance of Practical Cloud Computing Projects

ZenBot v2 exemplifies the value of hands-on cloud computing projects in university education. Cloud computing, as a discipline, is fundamentally practical in nature: the concepts of serverless architecture, distributed databases, managed API services, and cloud-native deployment patterns can be described theoretically but are only truly understood through direct application. Building a complete system that integrates multiple cloud services in a coherent architecture provides an experiential foundation that no amount of lecture material can replicate.

Furthermore, practical projects such as this one produce tangible artifacts, running systems and documented architectures, that serve as evidence of capability in professional contexts. The architectural patterns demonstrated in ZenBot v2, including serverless compute, NoSQL storage, API Gateway integration, and static web hosting, are precisely the patterns employed in real-world cloud application development across industries. Students who complete projects of this nature enter the workforce with a demonstrated ability to design and build cloud-native systems, a capability that is in significant demand in the current technology landscape.

In conclusion, ZenBot v2 represents a successful synthesis of cloud computing theory and practice, producing a functional, scalable, and educationally valuable platform that demonstrates the power of modern serverless cloud architecture. The project achieves its stated objectives,

addresses the limitations of its predecessor, and establishes a solid foundation for future enhancements that will further advance its capabilities as an educational AI tool.

References

[1] Amazon Web Services. (2024). AWS Lambda Developer Guide. Amazon Web Services Documentation. https://docs.aws.amazon.com/lambda/

[2] Amazon Web Services. (2024). Amazon DynamoDB Developer Guide. Amazon Web Services Documentation. https://docs.aws.amazon.com/dynamodb/

[3] Amazon Web Services. (2024). Amazon API Gateway Developer Guide. Amazon Web Services Documentation. https://docs.aws.amazon.com/apigateway/

[4] Amazon Web Services. (2024). Amazon S3 User Guide. Amazon Web Services Documentation. https://docs.aws.amazon.com/s3/

[5] Roberts, M. (2018). Serverless Architectures. Martin Fowler Blog. https://martinfowler.com/articles/serverless.html

[6] Wikipedia Foundation. (2024). Wikipedia API Documentation. https://www.mediawiki.org/wiki/API:Main_page

[7] Mell, P., & Grance, T. (2011). The NIST Definition of Cloud Computing. NIST Special Publication 800-145. National Institute of Standards and Technology.

[8] Weizenbaum, J. (1966). ELIZA—A Computer Program for the Study of Natural Language Communication Between Man and Machine. Communications of the ACM, 9(1), 36-45.

[9] PyPDF2 Documentation. (2024). PyPDF2 Python Library. https://pypdf2.readthedocs.io/

[10] Bernstein, D. (2014). Containers and Cloud: From LXC to Docker to Kubernetes. IEEE Cloud Computing, 1(3), 81-84.

[11] Baldini, I., et al. (2017). Serverless Computing: Current Trends and Open Problems. Research Advances in Cloud Computing. Springer, Singapore.

[12] Lewis, J., & Fowler, M. (2014). Microservices. Martin Fowler Blog. https://martinfowler.com/articles/microservices.html